

---

# Using AI to Understand the Basics of STM32

Brian Casillas

August 12, 2026

Physics B.S. — Engineering Technician — R&D — Electronics — Embedded Systems

## TL;DR

This article explains how I was able to use AI as an aid to modify and understand LED output behavior using an STLINK-V3MINIE probe.

## Introduction

A couple of years ago, I experimented with an STLINK-V3MINIE programmer/debugger probe to flash firmware onto an STM32F303 microcontroller. The microcontroller was on a PCB that, among other things, controlled LED behavior. One of the LEDs was a green status light (we will call it  $\gamma$ ) on the board that blinked whenever powered. The other two were connected to external outputs, which I will note as alpha ( $\alpha$ ) and beta ( $\beta$ ). My experimentation at the time was limited to changing an integer in a line of code to alter the blink rate of  $\gamma$ . This was about two years ago.

A friend and colleague suggested a fun project to revisit this exercise, leaning heavily on AI to install the tool set and modify the code to blink  $\gamma$  again. I took the exercise a little further and also modified the LEDs coming from  $\alpha$  and  $\beta$ .

## Hardware and Software Setup

I was able to borrow an old PCB with the STM32F303 microcontroller, the programmer/debugger probe, and the firmware project containing all the code.

Two STMicroelectronics programs were used:

- **STM32CubeIDE** — used to open the firmware project, modify the source code, compile it, and build a new firmware file.
- **STM32CubeProgrammer** — used to connect to the STM32F303 through the STLINK-V3MINIE and flash the compiled firmware onto the PCB.

## Methods: What I Did

### Workflow

The workflow for this project became:

---

```

    Modify and save the source code in STM32CubeIDE
    |
    Build the project
    |
    Locate the newly generated Debug/LedCtrl.elf
    |
    Open STM32CubeProgrammer
    |
    Connect to the PCB through the STLINK-V3MINIE
    |
    Load LedCtrl.elf
    |
    Flash the firmware
    |
    Reset or power-cycle the PCB and observe the new behavior

```

## First Firmware Modification: Modifying $\gamma$

The first successful modification changed the blink rate of the small green LED on the PCB. To do so, I first needed to identify which file contained the relevant code. It turned out to be `main.cpp`, which I used to call `LedCtrl::Test::blink()`. This provided a convenient working environment for the project.

The firmware used the STM32 millisecond tick:

```
HAL_GetTick()
```

along with logic such as:

```
bool b = bool((tick >> 10) & 1) == 0;
Gpio::LED_GRN.write(b);
```

## Understanding the logic

- The right-shift operator (`>>`) divides the tick count by powers of two (`tick >> 10`  $\rightarrow 2^{10} = 1024$ ). We don't produce a result until we've reached a multiple of 1024 ticks:

| Expression                    | Approximate state duration |
|-------------------------------|----------------------------|
| <code>tick &gt;&gt; 8</code>  | 256 ms                     |
| <code>tick &gt;&gt; 9</code>  | 512 ms                     |
| <code>tick &gt;&gt; 10</code> | 1024 ms                    |
| <code>tick &gt;&gt; 11</code> | 2048 ms                    |

- (`& 1`) extracts only the least significant bit (the last binary digit), making the value alternate between 0 and 1. So every 1024 ms the LED state flips between on and off, thus  $\approx 2.048$  seconds per full blink cycle.

Reducing the shift value therefore makes the LED blink faster. This was the first confirmation that the complete firmware workflow was functioning.

## Building and Flashing the ELF File

After modifying the source code, the project is built in STM32CubeIDE using the hammer icon. A successful build generates a new ELF file in the project's `Debug` directory:

---

```
LedCtrl/  
  Debug/  
    LedCtrl.elf
```

An important practical lesson was to use the ELF file generated directly in the `Debug` folder after a successful build. Attempts to extract or otherwise obtain the ELF through the IDE did not reliably produce firmware that behaved like the newly built version. Using the freshly generated `Debug/LedCtrl.elf` consistently worked.

## $\alpha$ and $\beta$ Outputs

The firmware exposes separate PWM controls for the  $\alpha$  and  $\beta$  LED assemblies:

```
Gpio::LedPwmAlphaBlu  
Gpio::LedPwmAlphaGrn  
Gpio::LedPwmAlphaRed  
  
Gpio::LedPwmBetaBlu  
Gpio::LedPwmBetaGrn  
Gpio::LedPwmBetaRed
```

Initially these names suggested that the external LEDs might be RGB LEDs. Testing the hardware showed that this assumption was incorrect.

Individual PWM outputs were activated one at a time to determine what each output actually controlled.

For  $\alpha$ :

```
LedPwmAlphaBlu -> BLUE LEDs  
LedPwmAlphaGrn -> WHITE LEDs  
LedPwmAlphaRed -> not needed for the current effect
```

For  $\beta$ :

```
LedPwmBetaBlu -> BLUE LEDs  
LedPwmBetaGrn -> WHITE LEDs  
LedPwmBetaRed -> not needed for the current effect
```

This was an important lesson: variable names are useful clues, but the actual hardware behavior should be verified experimentally.

## Using $\gamma$ to Understand GPIO Inputs

Initially it was unclear which input corresponded to the physical switch being moved.

The onboard green LED was temporarily used as a visual indicator for each input. For example:

```
Gpio::LED_GRN.write(Gpio::LED_INJECT.read());
```

If moving the switch changed that input, the green LED should change state.

Several inputs were tested:

```
LED_INJECT  
LED_LOAD  
LED_SCAN  
LED_SPARE1
```

The successful test was:

---

```
Gpio::LED_GRN.write(Gpio::LED_SCAN.read());
```

The green LED switched on and off when the physical switch was moved. This experimentally established that `LED_SCAN` was the useful input for the switch in the current setup.

This was effectively a simple embedded debugging technique: use a known output as a visual logic probe to determine the state of an unknown input.

### $\alpha$ LED Behavior

After determining that `LED_SCAN` followed the physical switch, the  $\alpha$  output was first tested with solid colors:

```
one switch position -> solid blue
other switch position -> solid white
```

Once that worked, blinking was added using the STM32 tick:

```
bool alpha_blink_on = ((tick >> 9) & 1) == 0;
```

The final  $\alpha$  behavior became:

```
Switch position 1 -> BLUE blinking
Switch position 2 -> WHITE blinking
```

The selected color is turned on and off approximately every 512 ms when using `tick >> 9`.

### $\beta$ LED Behavior

The  $\beta$  output was modified to provide two modes depending on the switch position:

```
Switch position 1 -> solid blue
Switch position 2 ->
    blue fade up
    blue fade down
    white fade up
    white fade down
    repeat
```

Blue and white are controlled independently so that only one color is illuminated at a time.

During the blue portion:

```
Gpio::LedPwmBetaBlu = dim_val;
Gpio::LedPwmBetaGrn = 0;
Gpio::LedPwmBetaRed = 0;
```

During the white portion:

```
Gpio::LedPwmBetaBlu = 0;
Gpio::LedPwmBetaGrn = dim_val;
Gpio::LedPwmBetaRed = 0;
```

## Results

At the end of the experiment, the firmware provides three visible behaviors:

- $\gamma$  **LED:** continuously blinks as a status indicator
- $\alpha$  **output:** switch-selectable blinking blue or blinking white

- 
- $\beta$  **output:** switch-selectable solid blue or alternating blue/white fade

## What I Learned

This project introduced several basic embedded concepts without requiring a complete understanding of the entire firmware architecture:

- PWM (Pulse Width Modulation) values are controlling LED brightness, allowing effects such as fading rather than simply switching an LED fully on or off using GPIO (General-Purpose Input/Output).
- STM32CubeIDE is used to modify and compile firmware.
- STM32CubeProgrammer and the STLINK-V3MINIE are used to flash firmware onto the STM32F303.
- The freshly generated `Debug/LedCtrl.elf` is the firmware file used for flashing.
- `HAL_GetTick()` provides a millisecond-based timing reference.
- Bit shifts can provide simple timing intervals.
- GPIO `.read()` operations inspect the state of digital hardware inputs.
- GPIO `.write()` operations control digital hardware outputs.
- An infinite `while(1)` loop changes the normal execution flow of the program.
- Hardware mappings can be determined experimentally by changing one input or output at a time and observing the result.
- Header (`.h`) files declare interfaces and tell the compiler what functions, classes, and variables are available.
- C++ (`.cpp`) files contain source-code implementations that define what those functions do.
- Understanding of program workflows, particularly how `.cpp` functions call.

## Conclusion

This was a great project that helped deepen my understanding of embedded systems. Blinking an LED sounds trivial, but it's the "Hello, World!" of microcontrollers.

This project also reminded me how valuable my physics background has been. Whether I'm working with electronics, mechanics, or mass spectrometry, having that foundation helps me understand what's happening and gives me a starting point when learning something new.

It also reinforced how important collaboration is. Working with people from different disciplines, asking questions, and sharing ideas can make solving complex problems a lot easier. AI has become another useful tool in that process. It helped guide me through unfamiliar code and troubleshoot along the way, but ultimately I still had to test everything on the actual hardware and figure out what worked.

I'm excited to take what I learned here and apply it to my next project.